CS-354: Complier Construction

(Spring 2026)

Lecture 2

Adnan Rashid

Email: adnan.rashid@seecs.edu.pk

Web: https://adrashid.github.io/

Recap

- Introduction

Outline

- Interpreter Vs Compiler
- Development of Compilers
  - Historical Perspective
- Structure of Compilers

Lecture's Content

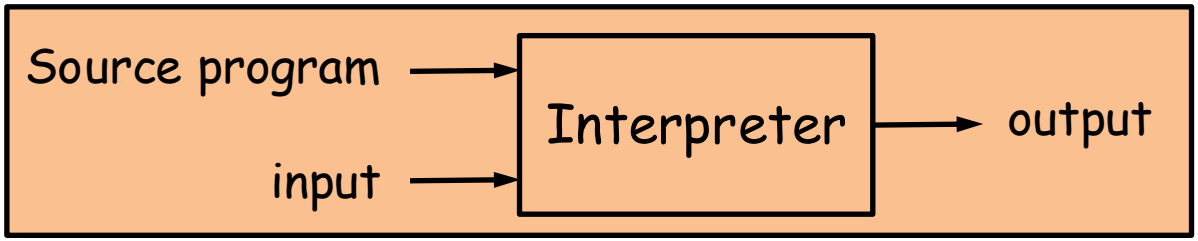
Compiler: Principles, Techniques and Tools (2nd Edition)

by

Alfred V. Aho et al.

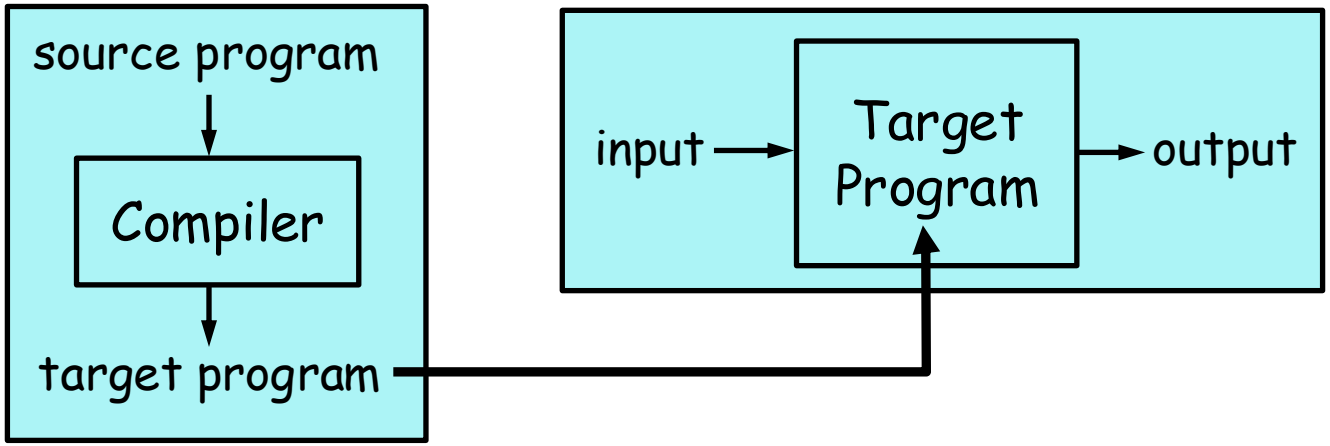
Section 1.1 (Page 2 onwards)

Interpreter Vs Compiler



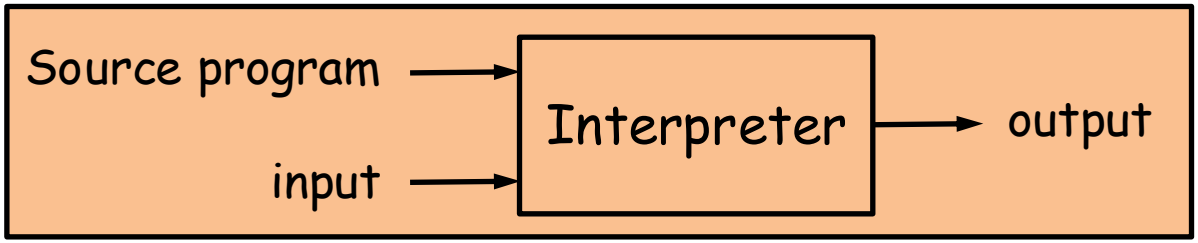
executes code at runtime
line by line

- An interpreter works online



- A compiler works offline

Interpreter Vs Compiler



- An interpreter reads a source program and produces the results of executing this source
  - Provides better error diagnostics but executes code slowly!

Interpreter Vs Compiler: Comparison Table

Aspect	Interpreter	Compiler
Execution	Online	Offline
Speed	Slower	Faster
Error reporting	Better, immediate	Usually all at once
Output	No executable	Executable file

Development of Compilers

- In 1954 IBM develops 704
  - Successor to 701 (1952)
- In early computers, SW cost was greater than HW cost
- Programing in assembly was slow and error-prone

An IBM 704 computer, with IBM 727 tape drives and IBM 780 CRT display. (Image courtesy of LLNL.)

Development of Compilers

- In 1953, John Backus introduced SpeedCoding
- It was an interpreter
- 300 bytes of memory
- 30% of IBM 701 memory

Backus in December 1989

FORTRAN 1

- FORTRAN = FORMULAS TRANSLATED
  - Developed 1954-57
  - 1958: 50% programs in FORTRAN
  - FORTRAN has the first successful compiler
  - Led to enormous body of theoretical work
  - Most compilers preserve FORTRAN 1 outlines

Why Compilation is Difficult?

- The Semantic Gap
  - The source program is structured into (depending on language) classes, functions, statements, expressions, . .
  - The target program is structured into instruction sequences, manipulating memory locations, stack and/or registers and with (conditional) jumps

Source Code:

```
z = 8*(x+5)-y
```

x86 Assembly:

```
movl  -12(%rbp), %eax
addl  $5, %eax
sall  $3, %eax
subl  -8(%rbp), %eax
movl  %eax, -4(%rbp)
```

- We need to structure the compilation process

Structure of Compiler

- Lexical Analysis
- Parsing
- Semantic Analysis
- Optimization
- Code Generation

Tip: It is a good idea to understand the functionality of compiler from the analogy of natural language

Compiler = language processing:
Letters → words → sentences → meaning → refined meaning → speech

Lexical Analysis

- First Step: recognize words (tokens)
  - Word (Token): Smallest meaningful unit above letters

You are not that irrational

Lexical Analysis

- First Step: recognize words (tokens)
  - Word (Token): Smallest meaningful unit above letters

arey ounot tha tir rational

arey ounot tha tir rational

- Without proper tokenization, the meaning breaks

Lexical Analysis

- Lexical Analysis divides program text into words or tokens (Converts characters into tokens)
  - Token: Smallest meaningful unit

if x == y then t=2; else t=3;

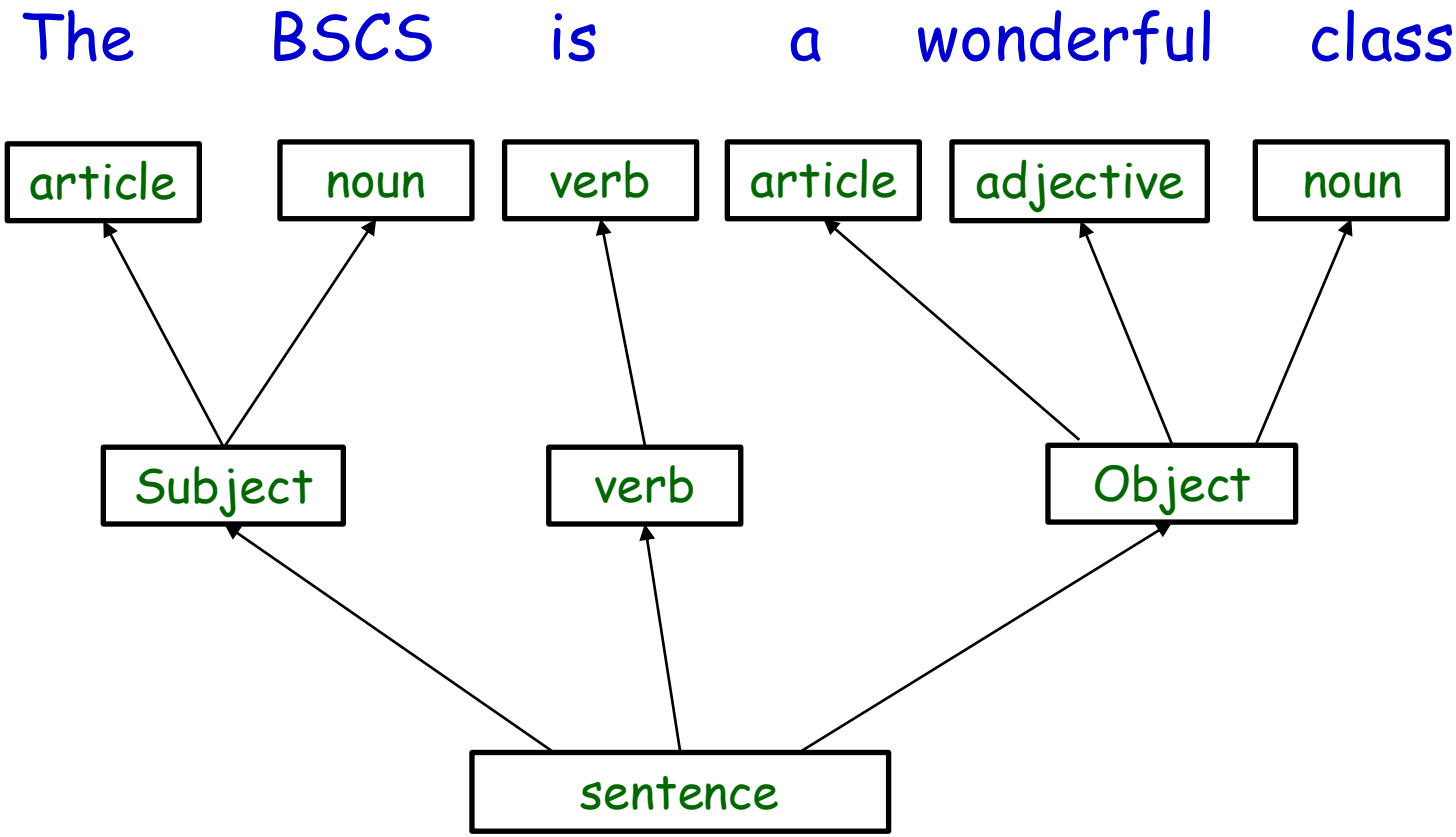
<u>if</u> x <u>==</u> y <u>then</u> t<u>=</u>2; else t=3;

Some tokens/words

Parsing

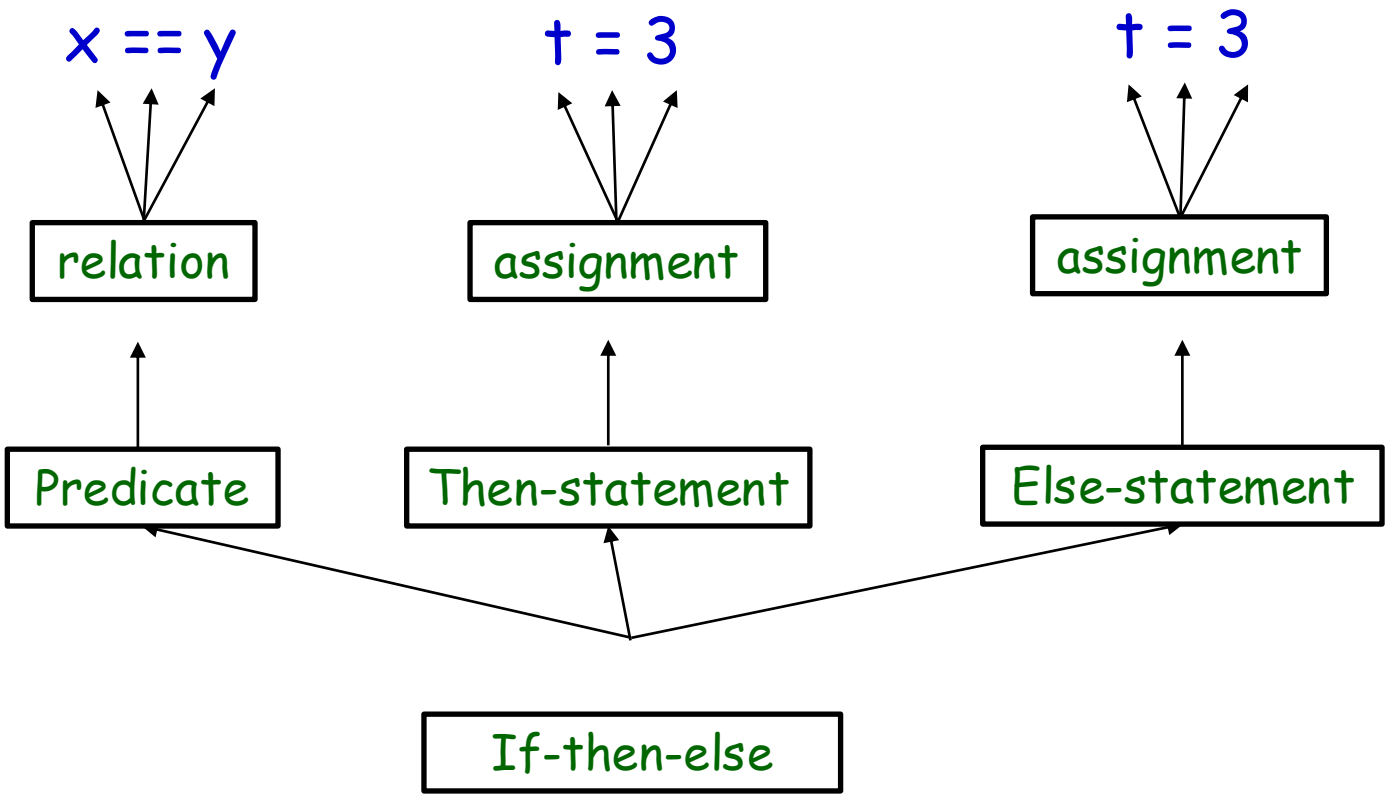
- Once words are understood, the next step is to understand the sentence structure
- Parsing = Diagramming sentences
  - The diagram is a tree, called parse tree

Parsing


The BSCS is a wonderful class
article
noun
verb
article
adjective
noun
Subject
verb
Object
sentence

Parsing

if x == y then t=2; else t=3;



Thank you!